

NumPy & Pandas

1. 넘파이와 판다스 라이브러리

넘파이와 판다스 라이브러리

● 넘파이의 배열

- 넘파이(NumPy): C 언어로 구현된 파이썬 라이브러리이며 숫자 데이터를 포함한 벡터와 행렬 연산에 유용
 - 설치되어 있지 않은 경우 pip 명령어로 넘파이 라이브러리를 설치하여 사용

```
pip install numpy;
```

- 배열: 넘파이의 핵심 객체. 같은 자료형인 데이터를 메모리에 물리적으로 연속 할당하여 인덱스로 데이터에 접근하기 편리

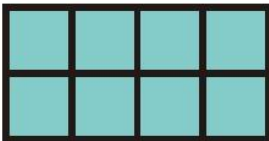
넘파이와 판다스 라이브러리

● 넘파이의 배열

• 넘파이 배열의 차원

- 데이터 분석에서 차원(Dimension)은 관측하고자 하는 데이터의 속성의 수 또는 측정 항목의 수를 의미
 - 스칼라(Scalar): 0차원 배열. 배열에서 값을 표현하는 가장 기본 단위이며, 스칼라에는 하나의 실수(Real number)를 담을 수 있음
 - 벡터(Vector): 1차원 배열. 스칼라(값) 여러 개를 나열한 튜플(Tuple)
 - 행렬(Matrix): 2차원 배열. 1차원 배열을 여러 개 묶은 배열
 - 텐서(Tensor): 벡터의 집합. 3차원 이상의 배열은 모두 텐서

넘파이 배열의 차원

차원	0차원	1차원	2차원	3차원 이상
구분	스칼라	벡터	행렬	텐서
공간 표현				

넘파이와 판다스 라이브러리

- 넘파이의 배열

- 넘파이 배열의 차원

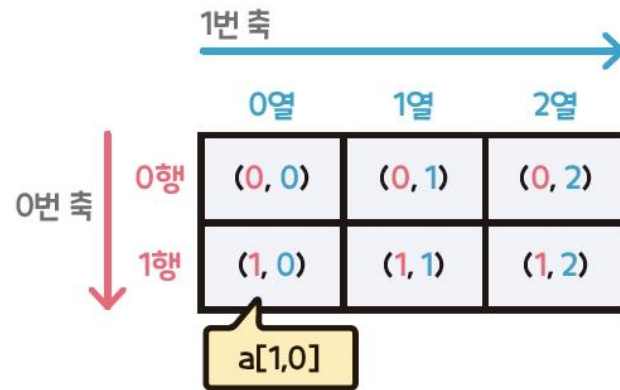
- 배열의 랭크(Rank)는 차원(Dimension)의 수이고, 모양(Shape)은 배열의 차원과 크기를 나타냄
 - ndarray: 넘파이의 다차원 배열(N-dimensional array) 객체
 - ① 자료형이 모두 같은 데이터를 담은 다차원 배열
 - ② 정수 또는 실수(부동 소수점 수)를 저장
 - ③ 배열 데이터에도 순서가 있으므로 인덱싱과 슬라이싱이 가능

넘파이와 판다스 라이브러리

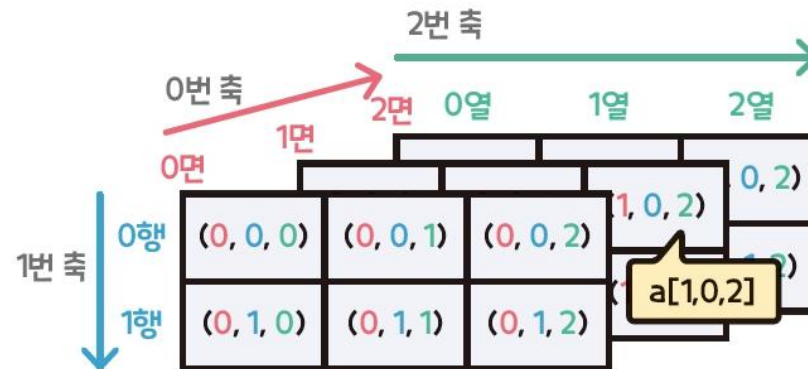
● 넘파이의 배열

• 넘파이 배열의 인덱싱과 축

- 배열을 정수나 다른 배열, 논리값, 음이 아닌 정수의 튜플(Tuple)로 인덱싱할 수 있음
- 배열이 2차원이면 대괄호 안에서 행 다음에 콤마(,)를 찍고 열의 인덱스를 붙임
- 3차원이면 면, 행, 열 순으로 인덱스를 붙임
- 배열의 축: 인덱스가 증가하는 방향. (a)에서 0번 축 방향으로 행 인덱스가 증가. (b)에서는 0번 축 방향으로 면 인덱스가 증가하며, 1번 축 방향으로 행 인덱스, 2번 축 방향으로 열 인덱스가 증가



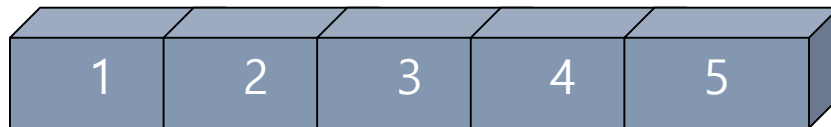
(a) 2차원 넘파이 배열의 인덱싱



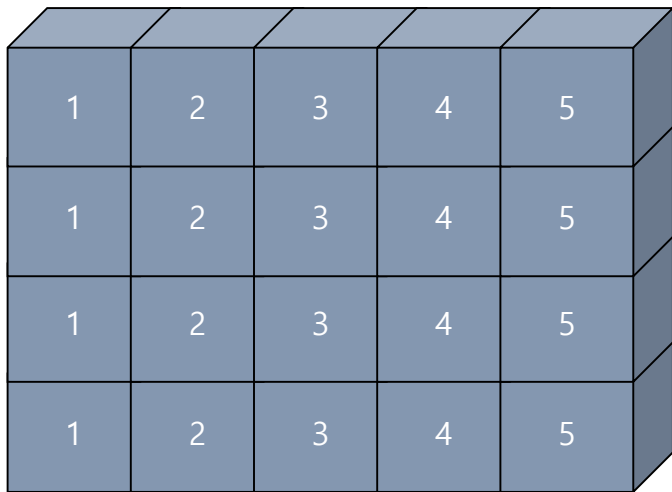
(b) 3차원 넘파이 배열의 인덱싱

넘파이와 판다스 라이브러리

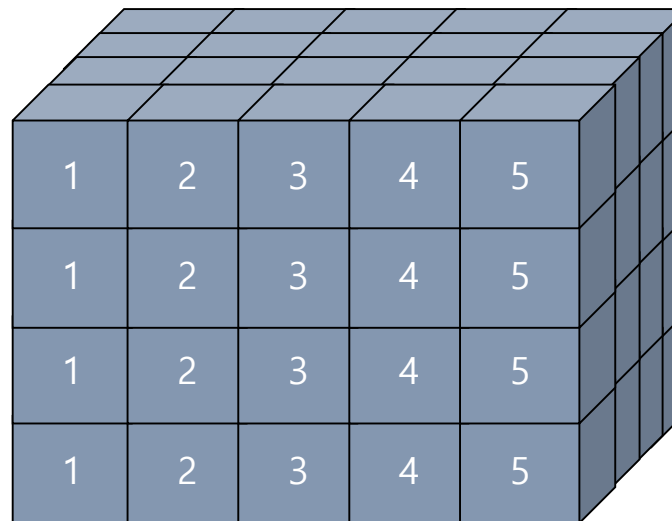
- 넘파이 ndarray 배열의 차원들



1차원 배열



2차원 배열



3차원 배열

넘파이와 판다스 라이브러리

- ndarray 배열의 shape 변수

- ndarray의 크기, 즉 행과 열의 수를 튜플 형태로 제공, 차원 확인까지 가능.

```
array1 = np.array([1,2,3])
```

```
print('array1 type:', type(array1))
```

```
print('array1 array 형태:', array1.shape)
```

```
array2 = np.array([[1,2,3], [2,3,4]])
```

```
print('array2 type:', type(array2))
```

```
print('array2 array 형태:', array2.shape)
```

```
array3 = np.array([[1,2,3]])
```

```
print('array3 type:', type(array3))
```

```
print('array3 array 형태:', array3.shape)
```


넘파이와 판다스 라이브러리

- ndarray 배열

- 차원 확인 : ndarray.ndim

```
print('array1: {:0}차원, array2: {:1}차원, array3: {:2}차원'.format(array1.ndim,array2.ndim,array3.ndim))
```

- 데이터 타입 확인 : ndarray.dtype

```
list1 = [1,2,3]  
print(type(list1))
```

```
array1 = np.array(list1)  
print(type(array1))  
print(array1, array1.dtype)
```

넘파이와 판다스 라이브러리

- ndarray 배열

- 서로 다른 데이터 타입의 ndarray에서의 처리

```
list2 = [1, 2, 'test']  
array2 = np.array(list2)  
print(array2, array2.dtype)
```

```
list3 = [1, 2, 3.0]  
array3 = np.array(list3)  
print(array3, array3.dtype)
```

- ndarray 내 데이터 값의 타입 변경 : astype()

```
array_int = np.array([1, 2, 3])  
array_float = array_int.astype('float64')  
print(array_float, array_float.dtype)
```

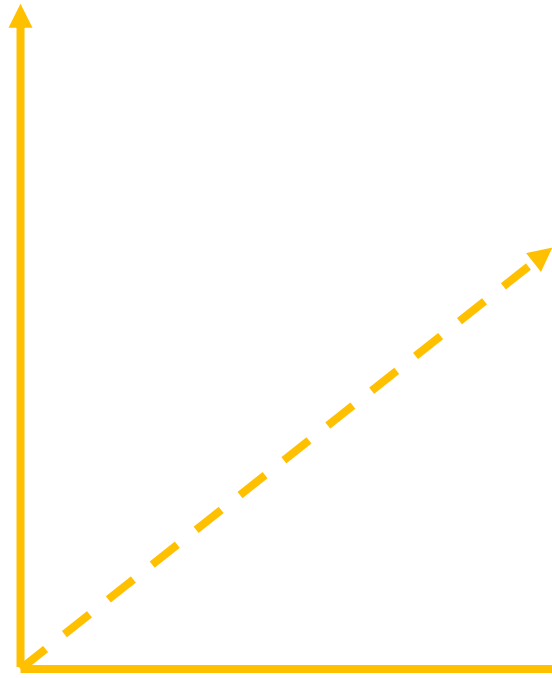
```
array_int1 = array_float.astype('int32')  
print(array_int1, array_int1.dtype)
```

```
array_float1 = np.array([1.1, 2.1, 3.1])  
array_int2 = array_float1.astype('int32')  
print(array_int2, array_int2.dtype)
```

넘파이와 판다스 라이브러리

- 3차원 배열의 축(axis)

y축(axis=2) : 열(column)



z축(axis=0) : depth(면)

x축(axis=1) : 행(row)

3차원 배열에서 axis(축)의 의미

넘파이와 판다스 라이브러리

- axis 이해

- numpy의 sum 함수 사용 예

- 다음 코드는 3차원 배열을 만들고, 3차원 배열의 합을 구하는 코드

```
>>> arr = np.arange(0, 2*3*4)
```

```
>>> len(arr)
```

```
24
```

```
>>> arr
```

```
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23])
```

```
>>> v = arr.reshape([2,3,4]) # 차원 변환 [2, 3, 4]; depth: 2, row: 3, column: 4
```

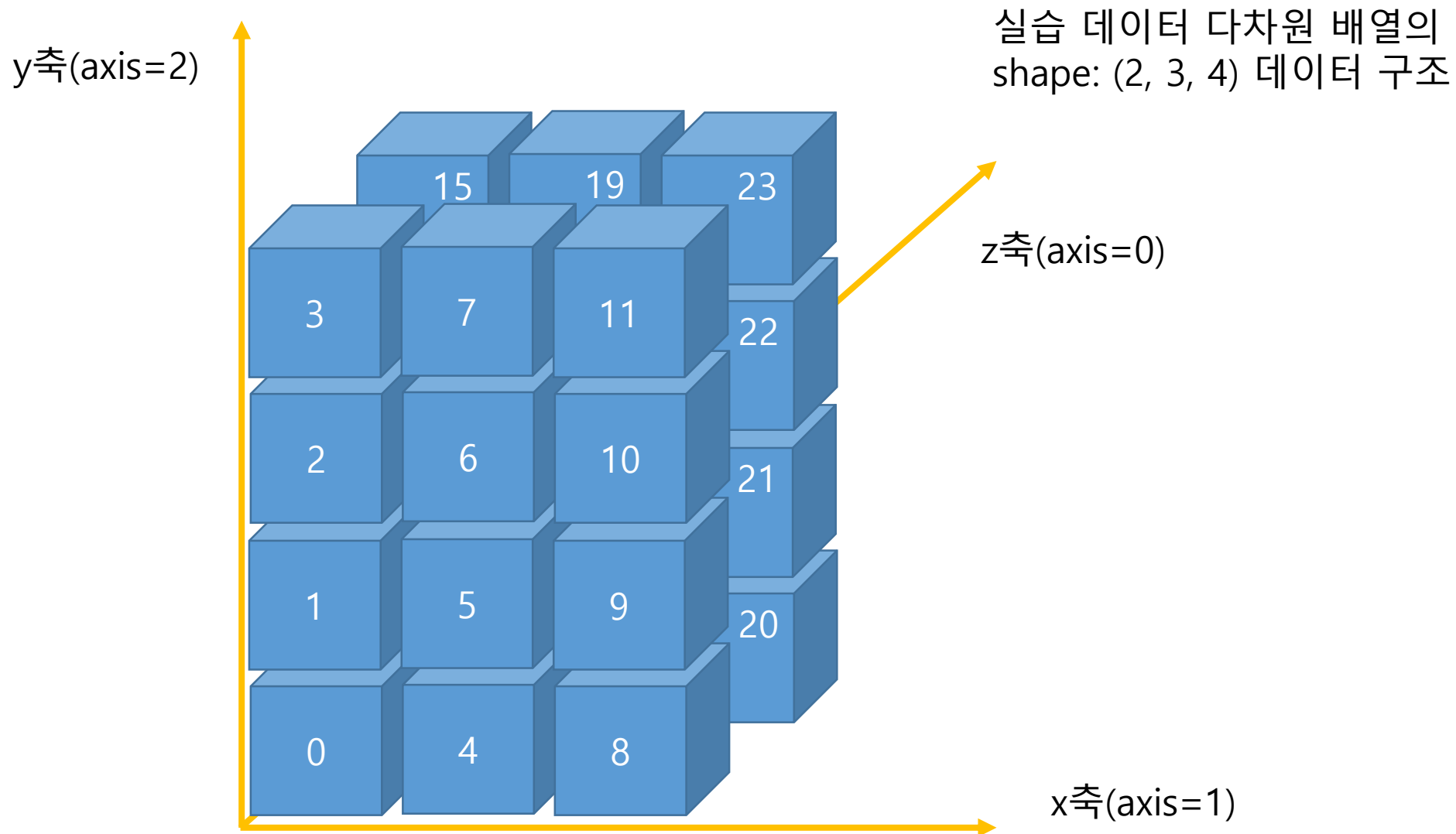
```
>>> v
```

```
array([[[ 0, 1, 2, 3],
        [ 4, 5, 6, 7],
        [ 8, 9, 10, 11]],
```

```
       [[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]])
```

넘파이와 판다스 라이브러리

- 3차원 배열의 축(axis) - 실습 데이터



넘파이와 판다스 라이브러리

● axis 이해

```
>>> v.shape # v의 형상
```

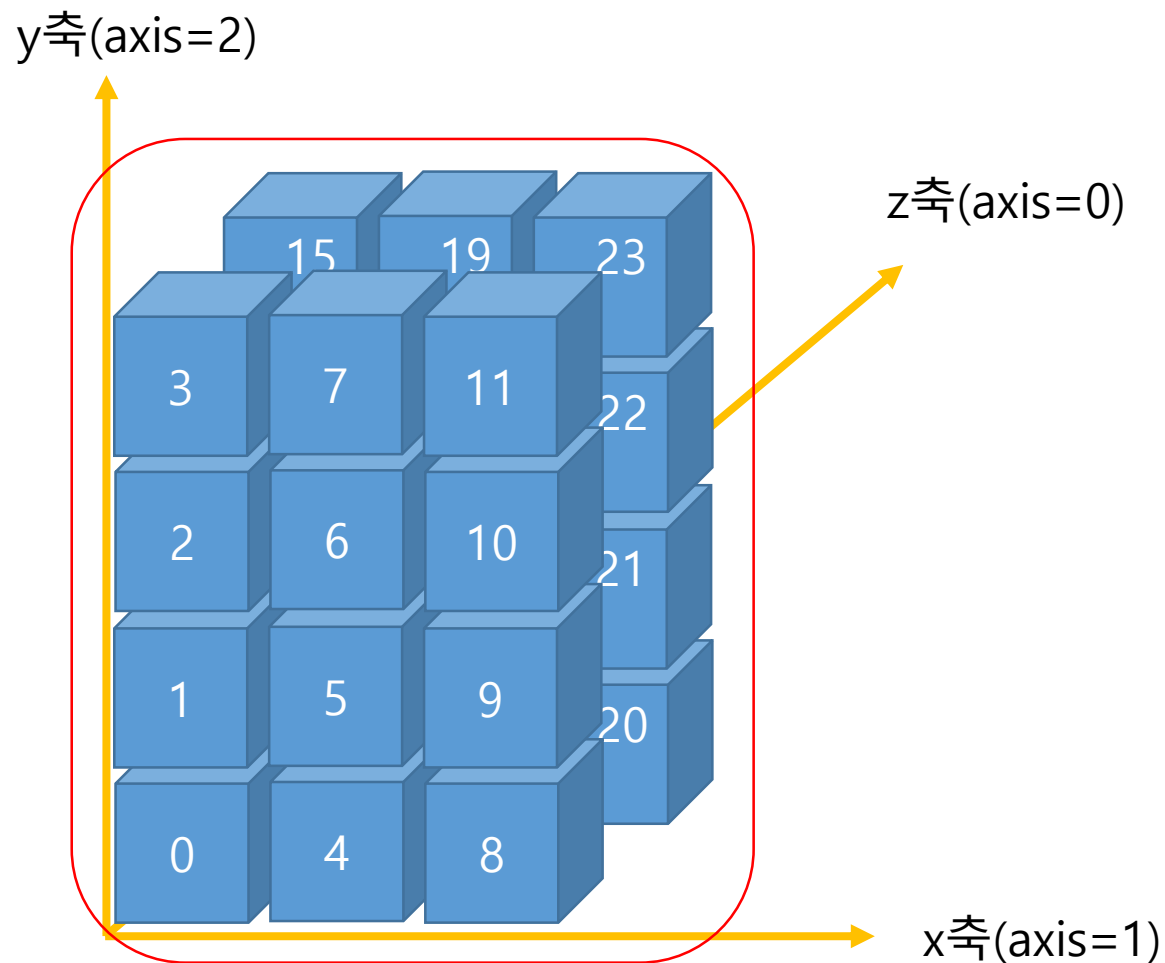
```
(2, 3, 4)
```

```
>>> v.ndim # v의 차원
```

```
3
```

```
>>> v.sum() # axis=None 기준 합계
```

```
276
```



넘파이와 판다스 라이브러리

● axis 이해

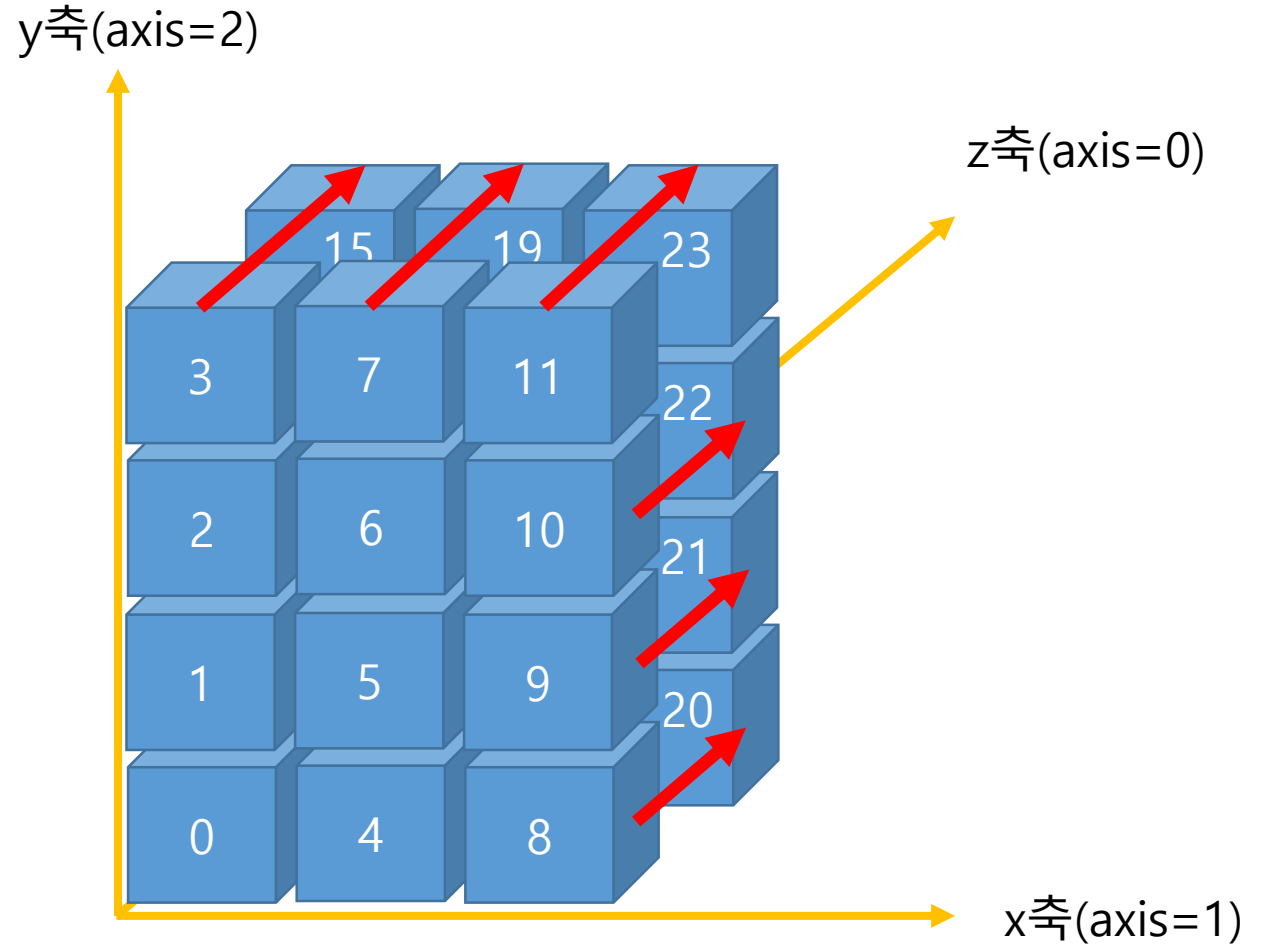
```
>>> res01=v.sum(axis=0) # axis=0 기준 합계
```

```
>>> res01.shape
```

```
(3, 4)
```

```
>>> res01
```

```
array([[12, 14, 16, 18],  
       [20, 22, 24, 26],  
       [28, 30, 32, 34]])
```



넘파이와 판다스 라이브러리

● axis 이해

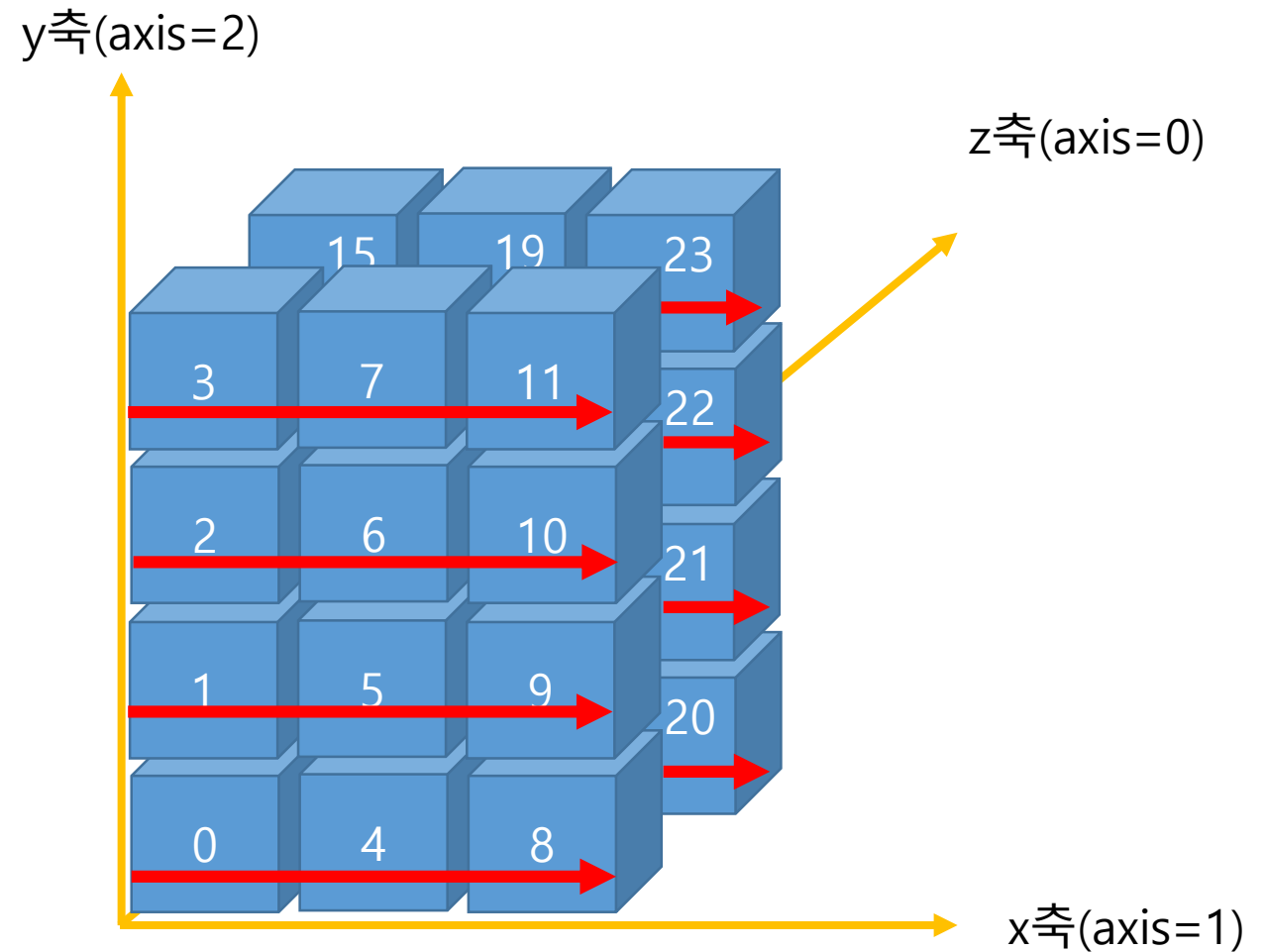
```
>>> res02=v.sum(axis=1) # axis=1 기준 합계
```

```
>>> res02.shape
```

```
(2, 4)
```

```
>>> res02
```

```
array([[ 12, 15, 18, 21],  
       [48, 51, 54, 57]])
```



넘파이와 판다스 라이브러리

● axis 이해

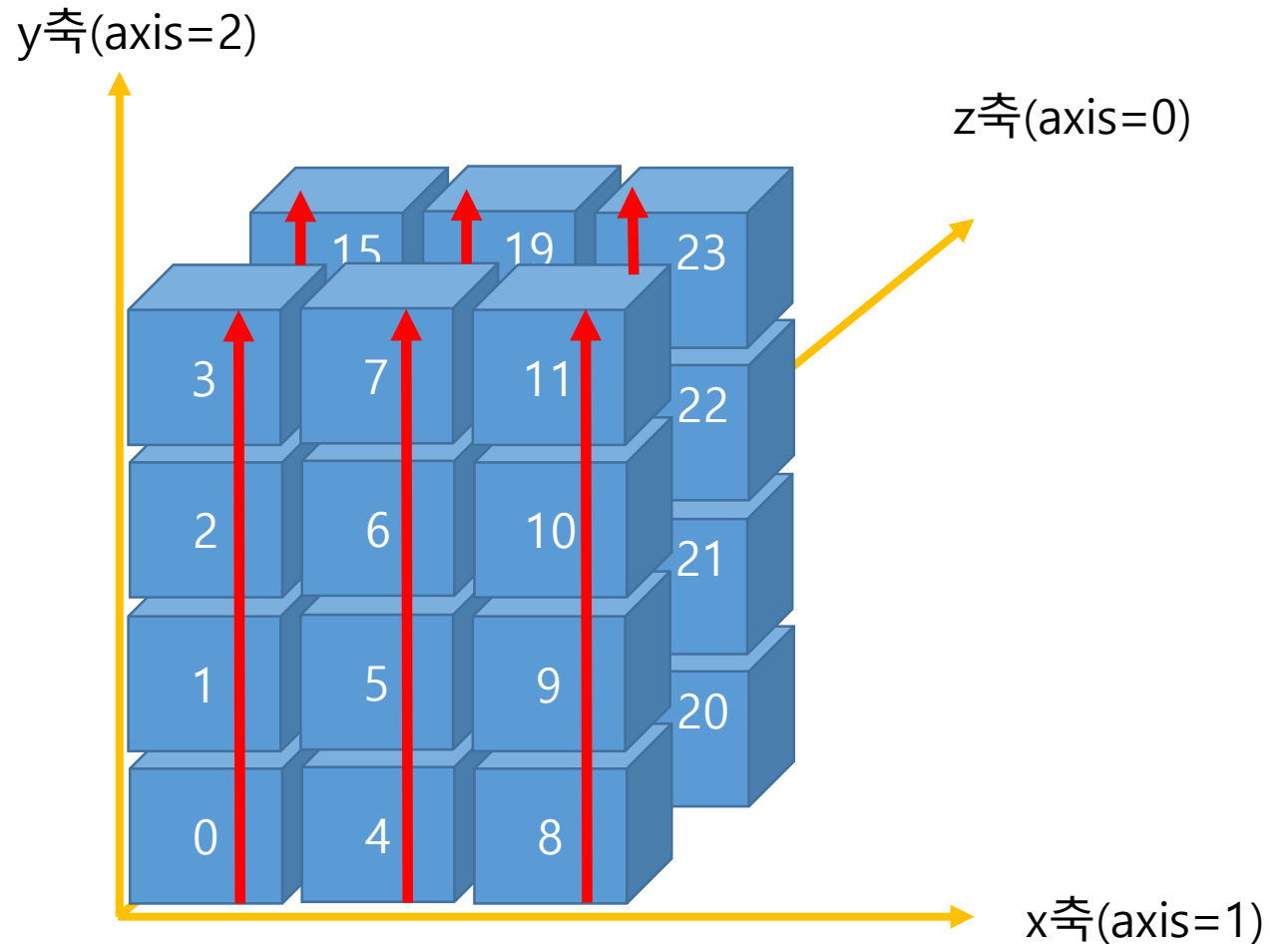
```
>>> res03=v.sum(axis=2) # axis=2 기준 합계
```

```
>>> res03.shape
```

```
(2, 3)
```

```
>>> res03
```

```
array([[ 6, 22, 38],  
       [54, 70, 86]])
```



넘파이와 판다스 라이브러리

- 넘파이의 배열
 - 넘파이 배열의 슬라이싱
 - 슬라이싱(Slicing): 배열에서 연속한 일부분을 잘라 선택하는 것
 - 슬라이스: 선택된 배열 조각
 - 튜플이나 리스트의 슬라이싱과 동일하게 인덱스와 콜론(:)을 사용
 - 배열이 2차원 이상일 때 행의 인덱스인지 열의 인덱스인지 주의해야 함

인덱싱과 슬라이싱

구분	종류	형식
인덱싱	논리값 인덱싱	조건 필터링 + 검색
	1차원	객체[a]
	2차원	객체[a,b]
슬라이싱	1차원	객체[a:b]
	2차원	객체[a:b, c:d]

넘파이와 판다스 라이브러리

● 넘파이의 배열

- 넘파이 배열의 슬라이싱
 - 콜론과 숫자를 함께 써서 처음과 끝 지점을 정함
 - 열이나 행, 면 전체를 선택하려면 숫자 대신 콜론 사용

	0	1	2
0	a[:2, :2]		
1			
2			

(a)

	0	1	2
0		a[:, 1:]	
1			
2			

(b)

	0	1	2
0	a[0]		
1			
2			

(c)

같은 슬라이스를
a[0, :], a[:, 0]로
표현 가능

	0	1	2
0			
1			
2		a[2, 1:]	

(d)

같은 슬라이스를
a[2, 1:]로
표현 가능

넘파이 배열의 슬라이싱

넘파이와 판다스 라이브러리

- 판다스의 시리즈와 데이터프레임

- 판다스(Pandas): 데이터프레임 자료구조를 제공하는 파이썬의 핵심 패키지
 - 설치되어 있지 않으면 명령어 pip를 사용

```
pip install pandas;
```

- 시리즈(Series): 인덱스와 값이 한 쌍을 이루는 1차원 자료구조 객체
 - 리스트는 값만 있고 인덱스가 0부터 자동 생성되는 반면, 시리즈는 사용자가 직접 인덱스를 정할 수 있음.

넘파이와 판다스 라이브러리

● 판다스의 시리즈와 데이터프레임

- 데이터프레임(Dataframe): 판다스의 기본 구조인 자료구조 객체
 - 시리즈 여러 개를 묶어서 데이터프레임을 만드므로 형태는 2차원 배열과 비슷
 - 데이터프레임은 행 인덱스, 열 이름(또는 열 인덱스), 값으로 구성
 - **행 인덱스**: 가로줄인 행(Row)을 구분하는 고유한 인덱스
 - **열 이름**: 세로줄인 열(Column)을 열 이름으로 구분
 - **값(Value)**: 행과 열이 교차하는 곳에 저장되는 데이터

	이름	성별	나이	키
0	한빛	남자	20	180
1	한결	남자	21	177
2	한라	여자	22	160

데이터프레임

넘파이와 판다스 라이브러리

● 넘파이와 판다스 비교

• 넘파이의 특징

- **다차원 배열 객체:** 객체 ndarray를 사용하여 다차원 배열을 생성하고 관리하는 기능이 넘파이의 핵심
- **정교한 브로드캐스팅:** 브로드캐스팅이란 넘파이에서 모양(Shape)이 서로 다른 배열끼리 연산하는 방식
- **C, C++, 포트란 코드를 통합:** 넘파이 내부의 복잡한 알고리즘이나 고성능이 필요한 부분은 C언어나 C++, 포트란(Fortran) 코드로 구현되어 있음
- **수학적 알고리즘 제공:** 넘파이에서는 선형대수의 함수, 푸리에 변환, 난수 기능 등 수학의 다양한 알고리즘을 제공

넘파이와 판다스 라이브러리

- 넘파이와 판다스 비교

- 판다스의 특징

- **대용량 데이터 처리:** 판다스를 활용하면 시리즈와 데이터프레임 자료구조로 대용량 데이터를 빠르게 처리
 - **시각적으로 알아보기 편리한 표 형태:** 데이터 구조가 표 형태이기 때문에 사용자가 데이터를 알아보기에 편리
 - **데이터 분석 도구 제공:** 판다스에서 제공하는 기능 중 데이터 분석에 자주 사용하는 기능은 결측치 처리, 관계 연산, 시계열

2. 넘파이 활용

넘파이 활용

- 넘파이의 배열 생성

- ndarray를 만들 때 넘파이 라이브러리에 정의된 `array()` 함수를 사용

```
a.shape: (4,)  
a[0]: 1  
b.shape: (2,3)  
b[0,0]: 1  
b[0]: [1, 2, 3]
```

배열 a		배열 b			
	0	0	1	2	
0	1	1	2	3	넘파이 배열의 예
1	2	4	5	6	
2	3				
3	4				

- 리스트 `list1`을 배열로 변환하여 변수 `a`에 할당, 배열 `b`는 요소를 바로 `array()` 함수의 인자로 작성하여 생성

넘파이 활용

- 넘파이의 배열 생성
 - 배열 생성 함수

넘파이 배열을 생성하는 함수	
함수	설명
array()	리스트를 배열로 변환
arange()	일정한 간격의 수를 ndarray 배열로 반환(파이썬의 range 함수와 유사함)
ones()	1로 채운 n차원 배열을 생성
zeros()	0으로 채운 n차원 배열을 생성
empty()	초기화하지 않은 빈 n차원 배열을 생성
eye() 또는 identity()	대각선 요소에만 1을 채우고 그 외에는 0으로 채워 2차원 배열을 생성
linespace()	초깃값부터 최종값까지 지정한 간격의 수를 채워 배열을 생성
full()	지정한 모양에 지정한 값으로 채운 배열을 생성

넘파이 활용

- 넘파이의 배열 생성

- 넘파이 자료형

- array() 함수로 배열을 생성할 때 dtype 옵션을 추가하여 배열의 넘파이 자료형을 지정할 수 있음
 - 배열 객체에 dtype 속성을 사용하면 넘파이 자료형을 확인할 수 있음

```
#실수형 배열 생성하기
```

```
a = np.array([1, 2], dtype=np.float64)  
print(a.dtype)
```

```
#정수형 배열로 변환하기
```

```
a_i8 = a.astype(np.int8)  
print(a_i8.dtype)
```

```
float64
```

```
int8
```

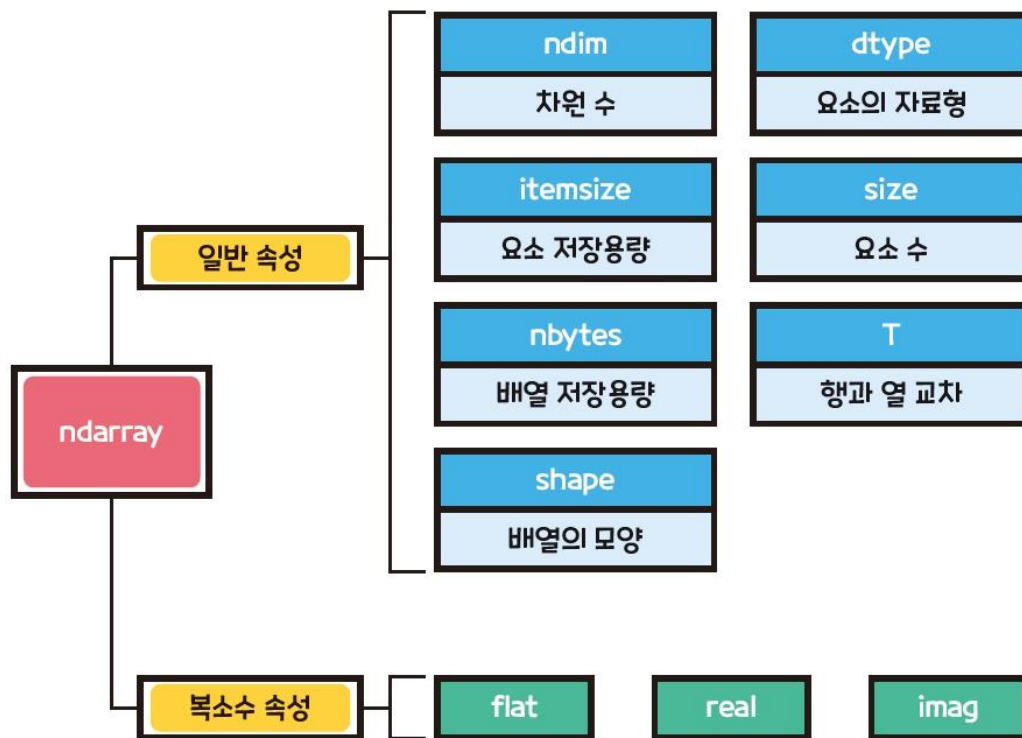
- 배열을 생성하면서 dtype으로 자료형을 지정

넘파이 활용

- 넘파이의 배열 생성

- 넘파이 배열의 속성

- 다차원 배열 객체 ndarray는 아래와 같은 속성을 가짐



ndarray의 속성

넘파이 활용

● 넘파이의 배열 생성

• 넘파이 배열의 속성

- ndim 속성: 'number of dimensions'의 약자로, 배열의 차원을 나타냄
- itemsize 속성: 요소의 바이트 수
 - size 속성: 요소의 개수
 - nbytes 속성: 배열 전체의 바이트 수
- Shape는 배열의 모양을 나타내는 속성
- 2차원 배열의 T 속성은 행과 열을 바꾼 교체 배열
- 속성을 호출하려면 객체 뒤에 속성 이름을 점으로 연결

객체.속성

넘파이 활용

- 넘파이의 배열 생성

- 배열의 모양

- 넘파이 배열의 전체 요소 개수를 유지하면서 모양을 변경할 수 있음

배열 모양을 변경하는 함수

함수	설명
<code>flatten()</code>	1차원 배열로 변경
<code>resize(i, j)</code>	배열의 모양을 $i \times j$ 로 변경
<code>transpose()</code>	열과 행을 교차

- 배열의 `shape` 속성에 튜플을 할당하여 모양을 지정할 수 있음

```
객체.shape = (행 크기, 열 크기)
```

넘파이 활용

- 넘파이의 배열 생성
 - 배열의 모양
 - `flatten()` 함수: 다차원 배열을 1차원 배열로 변경
 - `resize()` 함수: 튜플을 입력하면 배열의 모양을 변경
 - `transpose()` 함수는 속성 T와 마찬가지로 행렬의 행과 열을 교차

넘파이 활용

- 넘파이 배열 다루기

- 마스크

- 논리값 인덱싱(Boolean indexing) 또는 마스크(Masking) : 조건에 맞는 값을 출력
 - True와 False 대신 1과 0로 마스크 배열 mask를 만들
 - 넘파이의 `random.randn( )` 함수는 평균이 0이고, 표준편차가 1인 난수를 생성
 - 조건식 마스크로 데이터에서 기준보다 큰 값이나 작은 값을 쉽게 찾을 수 있음

넘파이 활용

- 넘파이 배열 다루기
 - 유니버설 함수와 브로드캐스팅
 - 유니버설 함수: 배열 요소끼리의 기본 연산을 수행하는 함수

넘파이 연산 함수

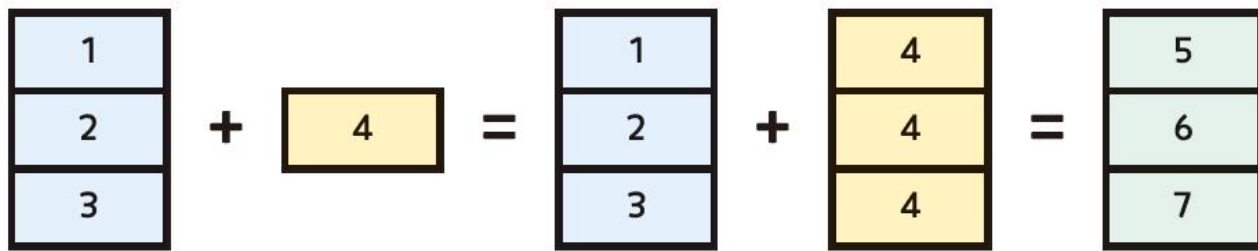
함수	설명	사용법
abs()	원소의 절댓값을 반환	numpy.abs(arr)
fabs()	실수 원소의 절댓값을 반환	numpy.fabs(arr)
sqrt()	원소의 제곱근을 반환	numpy.sqrt(arr)
square()	원소의 제곱을 반환	numpy.square(arr)
exp()	원소의 지수를 반환	numpy.exp(arr)
log()	원소에 밑이 e인 로그를 취하여 반환	numpy.log(arr)
add()	두 배열 원소의 합을 반환	numpy.add(arr1, arr2)
subtract()	두 배열 원소의 차를 반환	numpy.subtract(arr1, arr2)
multiply()	두 배열 원소의 곱을 반환	numpy.multiply(arr1, arr2)
divide()	두 배열 원소의 나눗셈 결과를 반환	numpy.divide(arr1, arr2)
floor_divide()	나눗셈의 정수 몫을 반환	numpy.floor_divide(arr1, arr2)
mod()	두 배열 원소 나눗셈의 정수 나머지를 반환	numpy.mod(arr1, arr2)

넘파이 활용

● 넘파이 배열 다루기

- 유니버설 함수와 브로드캐스팅

- 서로 다른 형태의 배열을 유니버설 함수로 연산할 때 브로드캐스팅이 일어남
- 브로드캐스팅(Broadcasting): 둘 중 작은 차원인 배열을 변형하여 큰 차원의 배열에 맞춘 다음 두 배열을 요소별로 연산하는 동작



넘파이 활용

● 넘파이 배열 다루기

• 배열 복사

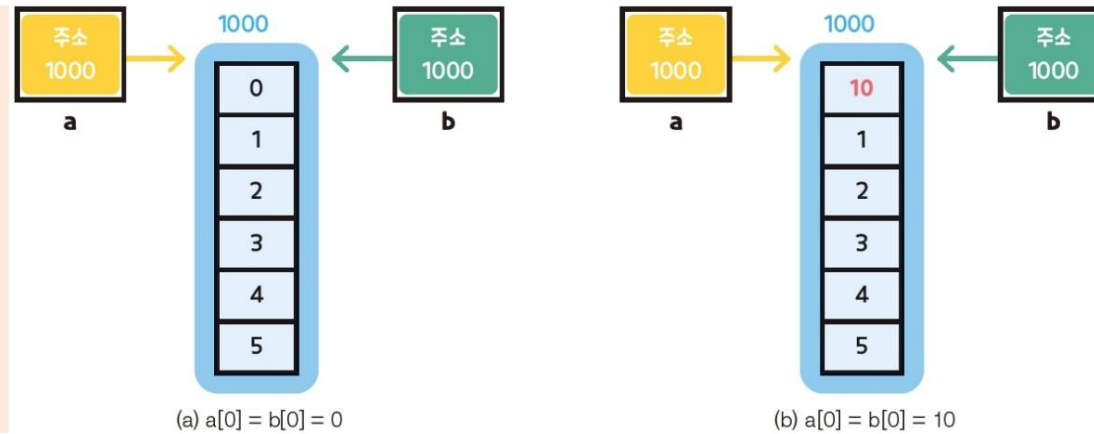
- 얇은 복사: 등호(=)로 복제. 원본 데이터의 주소가 복사됨
- 깊은 복사: copy() 함수를 사용. 배열 복사본을 생성함. 원본을 수정해도 복사본이 바뀌지 않음

얇은 복사

```
a = np.arange(6)
b = a
print(a)
print(b is a)
```

```
b[0] = 10
print(a)
```

```
[0 1 2 3 4 5]
True
[10 1 2 3 4 5]
```

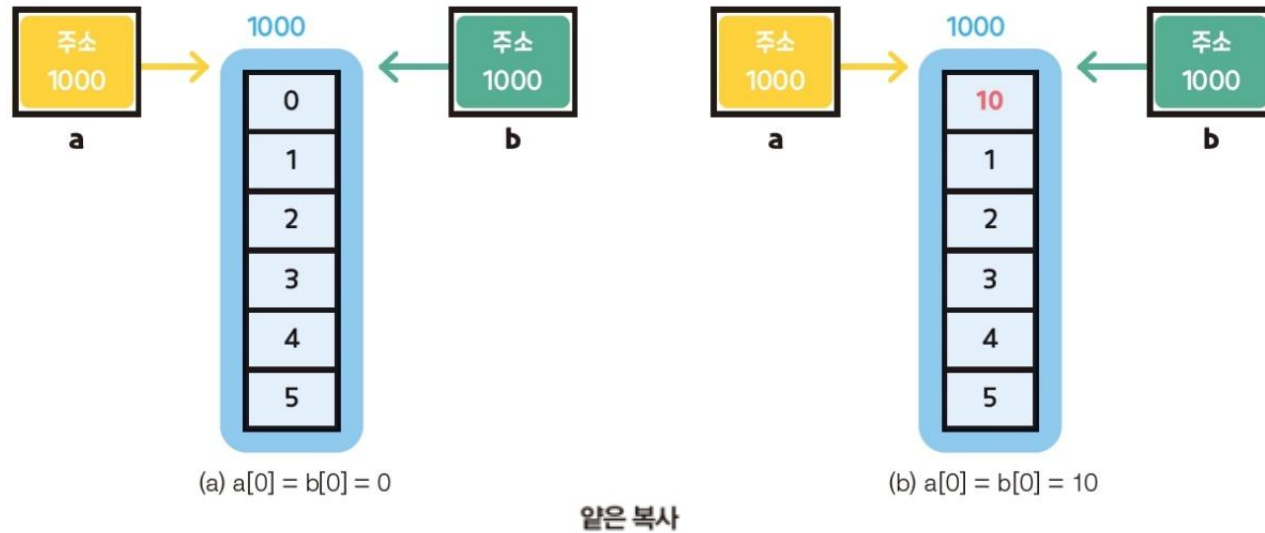


- 복사본인 배열 b의 값만 바꾸어도 원본 배열 a의 요소가 함께 변경됨

넘파이 활용

- 넘파이 배열 다루기

- 배열 복사



- $a[0]$ 은 변수 a 가 가리키는 배열 중 첫 번째 요소를 의미함
- a 의 값을 b 에 얕은 복사로 저장했으므로 b 도 같은 배열을 가리키게 됨
- $b[0]$ 에 10을 할당하여 배열 원소를 변경하면 같은 주소를 가리키는 $a[0]$ 의 값도 10으로 변경됨

넘파이 활용

- 넘파이 배열 다루기

- 배열 복사

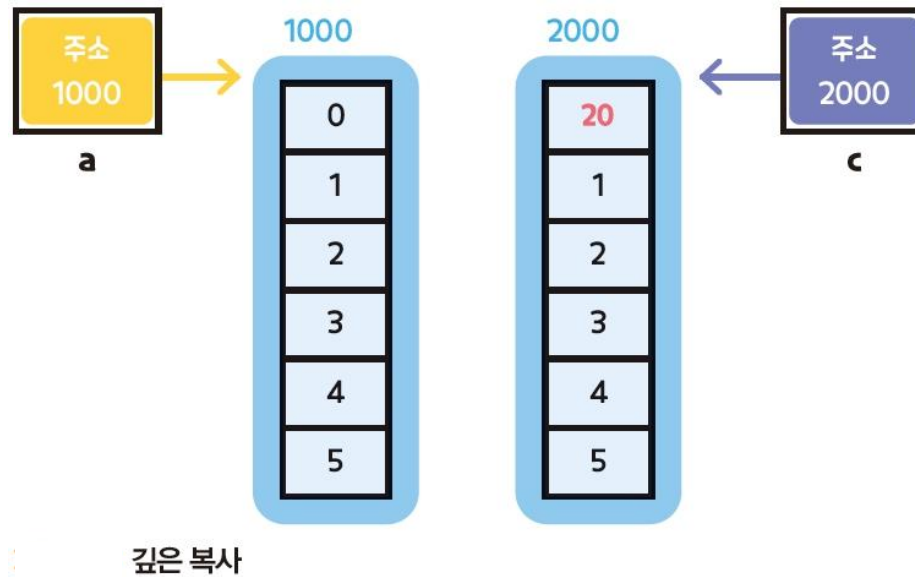
깊은 복사

```
a = np.arange(6)
c = a.copy( )

c[0] = 20
print('A: ', a)
print('C: ', c)
```

실행 결과

```
A: [0 1 2 3 4 5]
C: [20 1 2 3 4 5]
```

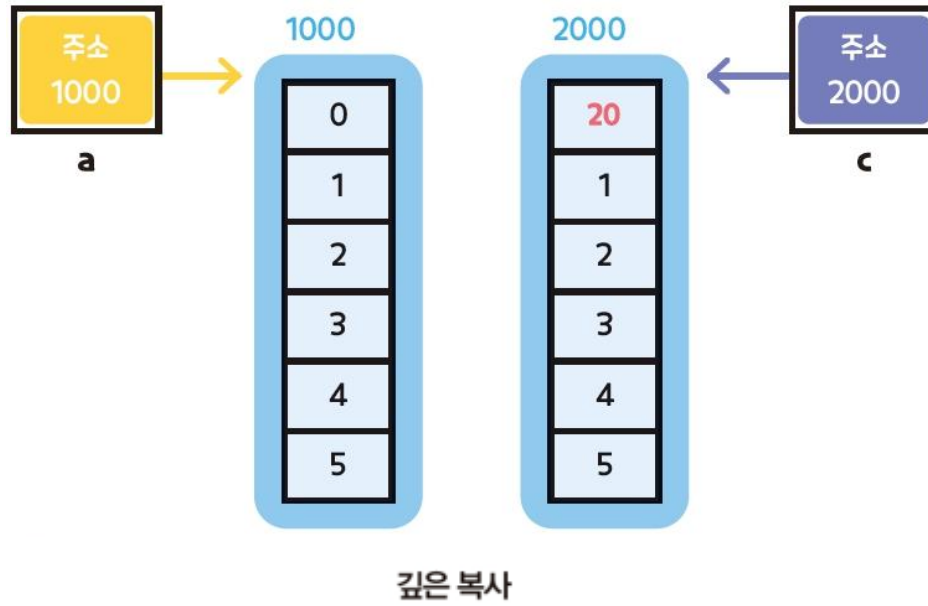


- copy() 함수를 사용하면 복사본인 배열 c의 값을 수정해도 원본 a에 영향이 없음

넘파이 활용

- 넘파이 배열 다루기

- 배열 복사



- c[0]에 20을 할당하더라도 a가 가리키는 배열과는 별개이므로 a[0]은 그대로 0

넘파이 활용

- 넘파이 배열 다루기

- 배열 정렬

정렬 함수

함수	설명
<code>np.sort(배열)</code>	배열을 정렬해서 반환, 원본을 유지함
<code>배열.sort()</code>	배열 정렬, 원본을 정렬함
<code>np.argsort(배열)</code>	정렬된 배열의 원래 인덱스를 반환

- `np.sort( )` 함수는 원본 배열 `a`의 요소 순서가 유지되고, `sort( )` 함수는 원본을 정렬

3. 판다스 활용

판다스 활용

● 시리즈와 데이터프레임 생성

- 판다스의 Series() 함수에 리스트를 입력하여 리스트 데이터로 시리즈를 생성

리스트로 시리즈 생성

```
import pandas as pd

a = pd.Series([1, 2, 3, 4])
print(a)
b = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
print(b)
```

```
0 1
1 2
2 3
3 4
dtype: int64
a 1
b 2
c 3
dtype: int64
```

- 시리즈 a는 인덱스를 지정하지 않았으므로 기본 인덱스로 숫자 0, 1, 2, 3이 지정됨
- 시리즈 b는 값 1, 2, 3에 인덱스로 문자열 'a', 'b', 'c'를 지정

판다스 활용

● 시리즈와 데이터프레임 생성

- 이번에는 판다스의 DataFrame() 함수로 데이터프레임을 만들기
- 파이썬의 리스트, 파이썬의 딕셔너리, 넘파이의 배열을 활용
- 리스트를 만들어 데이터프레임의 데이터로 한 행씩 지정할 수 있음

```
pd.DataFrame(값, columns=열 이름 리스트)
```

리스트로 데이터프레임 생성

```
list1 = list(['한빛', '남자', '20', '180'],  
            ['한결', '남자', '21', '177'],  
            ['한라', '여자', '20', '160']))  
col_names = ['이름', '성별', '나이', '키']  
pd.DataFrame(list1, columns=col_names)
```

	이름	성별	나이	키
0	한빛	남자	20	180
1	한결	남자	21	177
2	한라	여자	22	160

판다스 활용

● 시리즈와 데이터프레임 생성

- 데이터프레임을 생성할 때 딕셔너리를 사용하면 키는 열 이름이 되고 값은 열에 대한 각 행의 데이터이므로 리스트 형식

```
pd.DataFrame({키1:{인덱스0:값1, 인덱스1:값2}, 키2:{인덱스0:값3, 인덱스1:값4}})
```

딕셔너리로 데이터프레임 생성

```
dict1 = {'이름':{0:'한빛', 1:'한결', 2:'한라'},  
        '성별':{0:'남자', 1:'남자', 2:'여자'},  
        '나이':{0:'20', 1:'21', 2:'20'},  
        '키':{0:'180', 1:'177', 2:'160'}}  
pd.DataFrame(dict1)
```

- 실행 결과로 생성되는 데이터프레임은 리스트 데이터프레임의 결과와 동일함

판다스 활용

● 시리즈와 데이터프레임 생성

- 실행 결과로 생성되는 데이터프레임은 리스트 데이터프레임 결과와 동일
 - 열 이름 리스트를 만들고 columns 옵션으로 데이터프레임의 열 이름으로 지정

배열로 데이터프레임 생성

```
import numpy as np

#배열
arr1 = np.array([[ '한빛', '남자', '20', '180'],
                  [ '한결', '남자', '21', '177'],
                  [ '한라', '여자', '20', '160']])

#열 이름 리스트
col_names = ['이름', '성별', '나이', '키']

#데이터프레임 생성하기
pd.DataFrame(arr1, columns=col_names)
```

판다스 활용

● 시리즈와 데이터프레임 생성

- CSV 파일로 데이터프레임 생성
 - 데이터프레임을 CSV로 저장하거나 CSV 파일을 데이터프레임 객체로 읽어옴

CSV 파일로 데이터프레임 생성

```
list1 = list(['허준호', '남자', '30', '183'],  
             ['이가원', '여자', '24', '162'],  
             ['배규민', '남자', '23', '179'],  
             ['고고림', '남자', '21', '182'],  
             ['이새봄', '여자', '28', '160'],  
             ['이보람', '여자', '26', '163'],  
             ['이루리', '여자', '24', '157'],  
             ['오다현', '여자', '24', '172']))  
col_names = ['이름', '성별', '나이', '키']  
df = pd.DataFrame(list1, columns=col_names)  
df
```

	이름	성별	나이	키
0	허준호	남자	30	183
1	이가원	여자	24	162
2	배규민	남자	23	179
3	고고림	남자	21	182
4	이새봄	여자	28	160
5	이보람	여자	26	163
6	이루리	여자	24	157
7	오다현	여자	24	172

판다스 활용

- 시리즈와 데이터프레임 생성
 - CSV 파일로 데이터프레임 생성

데이터프레임을 CSV 파일로 저장

```
#디렉토리에 csv 파일로 저장하기
df.to_csv('./file.csv', header=True, index=False, encoding='utf-8')
#CSV 파일 읽기
df2 = pd.read_csv('./file.csv', sep=',')

df2
```

판다스 활용

- 시리즈와 데이터프레임 생성

- 열 이름 조회

모든 열 이름 조회

```
df.columns
```

실행 결과

```
Index(['이름', '성별', '나이', '키'], dtype='object')
```

판다스 활용

● 시리즈와 데이터프레임 생성

- 열 이름 조회
 - describe() 함수로 열별 값의 개수, 빈도 수와 같은 통계를 확인.

데이터프레임 값 개수와 빈도수 확인

```
df.describe( )
```

실행 결과

	이름	성별	나이	키
count	8	8	8	8
unique	8	2	6	8
top	허준호	여자	24	183
freq	1	5	3	1

- count는 값의 개수, unique는 유일한 값의 개수, top은 제일 개수가 많은 값이고, freq는 빈도수(frequency)이므로 그 개수를 나타냄

판다스 활용

● 시리즈와 데이터프레임 생성

• 데이터 미리보기

- 데이터 미리보기 기능은 데이터의 일부만 보고 열이 어떻게 구성되어 있는지, 값의 범위는 어느 정도인지 확인할 때 사용
- `head()` 함수와 `tail()` 함수에 정수 `n`을 입력하면 각각 데이터프레임의 처음 `n`개 행, 마지막 `n`개 행을 반환

데이터프레임의 처음 세 행 검색

```
df.head(3)
```

	이름	성별	나이	키
0	허준호	남자	30	183
1	이가원	여자	24	162
2	배규민	남자	23	179

판다스 활용

- 시리즈와 데이터프레임 생성
 - 데이터 미리보기

데이터프레임의 마지막 다섯 행 검색

```
df.tail( )
```

	이름	성별	나이	키
3	고고림	남자	21	182
4	이새봄	여자	28	160
5	이보람	여자	26	163
6	이루리	여자	24	157
7	오다현	여자	24	172

판다스 활용

● 데이터프레임 데이터 분석

• 행 정렬

- 인덱스나 특정 열의 값을 기준으로 행을 정렬할 수 있음
- 기본 정렬 방향은 오름차순(ascending)이며 내림차순(descending) 정렬은 ascending 인자를 False로 지정

인덱스 기준으로 정렬

```
df.sort_index(axis=0).head( )
```

실행 결과

	이름	성별	나이	키
0	허준호	남자	30	183
1	이가원	여자	24	162
2	배규민	남자	23	179
3	고고림	남자	21	182
4	이새봄	여자	28	160

판다스 활용

● 데이터프레임 데이터 분석

- 행 정렬
 - `sort_values()` 함수를 이용하여 특정 열을 기준으로 정렬

열 기준으로 정렬

#나이 열과 키 열을 기준으로 행을 정렬하기

```
df.sort_values(by=['나이', '키'], ascending=False).head( )
```

실행 결과

	이름	성별	나이	키
0	허준호	남자	30	183
4	이새봄	여자	28	160
5	이보람	여자	26	163
7	오다현	여자	24	172
1	이가원	여자	24	162

판다스 활용

● 데이터프레임 데이터 분석

• 데이터 조회

- 데이터를 열이나 조건식을 기준으로 조회할 수 있음

```
데이터프레임[조건식]
```

```
#시리즈로 출력하기
```

```
데이터프레임[[조건식]]
```

```
#데이터프레임으로 출력하기
```

- 조건식 자리에 리스트 형식으로 원하는 열 이름을 적기

열 이름으로 데이터 조회

```
df[['이름', '키']].head( )
```

	이름	키
0	허준호	183
1	이가원	162
2	배규민	179
3	고고림	182
4	이새봄	160

판다스 활용

- 데이터프레임 데이터 분석

- 데이터 조회

- `iloc` 함수를 사용하여 인덱스로 데이터를 조회할 수 있음

```
df.iloc[행 인덱스, 열 이름]
```

인덱스로 데이터 조회

```
df.iloc[1:4, 0:3]
```

실행 결과

	이름	성별	나이
1	이가원	여자	24
2	배규민	남자	23
3	고고림	남자	21

- 두 번째 행부터 네 번째 행의 첫 번째 열부터 세 번째 열을 조회

판다스 활용

● 데이터프레임 데이터 분석

• 데이터 조회

- 키가 180보다 큰 사람만 조회

조건식을 만족하는 데이터 조회(1)

```
df[df['키'] > 180].head( )
```

실행 결과

```
-----  
TypeError Traceback (most recent call last)  
<ipython-input-16-c264ba620ede> in <module>  
----> 1 df[df['키'] > 180]  
TypeError: '>' not supported between instances of 'str' and 'int'
```

- 오류는 키 열의 데이터 자료형이 문자열이기 때문

판다스 활용

- 데이터프레임 데이터 분석

- 데이터 조회

조건식을 만족하는 데이터 조회(2)

```
list1 = list(['허준호', '남자', 30, 183],  
            ['이가원', '여자', 24, 162],  
            ['배규민', '남자', 23, 179],  
            ['고고림', '남자', 21, 182],  
            ['이새봄', '여자', 28, 160],  
            ['이보람', '여자', 26, 163],  
            ['이루리', '여자', 24, 157],  
            ['오다현', '여자', 24, 172]))
```

```
col_names = ['이름', '성별', '나이', '키']  
df = pd.DataFrame(list1, columns=col_names)
```

```
df[df['키'] > 180]
```

	이름	성별	나이	키
0	허준호	남자	30	183
3	고고림	남자	21	182

- 키 열의 데이터를 숫자형으로 입력하면 조건식으로 데이터를 조회할 수 있음

판다스 활용

● 데이터프레임 데이터 분석

• 데이터 조회

- 조건식에 `isin()` 함수, `and` 기호(`&`), `or` 기호(`|`), `str.contains()` 함수를 사용하여 더 복잡한 조건으로 조회할 수도 있음

리스트 요소와 일치하는 데이터 조회

```
df[df['나이'].isin([21, 23])]
```

실행 결과

	이름	성별	나이	키
2	배규민	남자	23	179
3	고고림	남자	21	182

- 나이가 21세 또는 23세인 사람을 조회

판다스 활용

- 데이터프레임 데이터 분석

- 데이터 조회

두 조건식을 동시에 만족하는 데이터 조회

```
df[(df['성별'] == '여자') & (df['키'] > 160)]
```

실행 결과

	이름	성별	나이	키
1	이가원	여자	24	162
5	이보람	여자	26	163
7	오다현	여자	24	172

- '여자'이면서 키가 160보다 큰 사람을 조회

판다스 활용

- 데이터프레임 데이터 분석

- 데이터 조회

두 조건식 중 하나 이상 만족하는 데이터 조회

```
df[(df['나이'] >= 28) | (df['성별'] == '남자')]
```

실행 결과

	이름	성별	나이	키
0	허준호	남자	30	183
2	배규민	남자	23	179
3	고고림	남자	21	182
4	이새봄	여자	28	160

- 28세 이상이거나 '남자'인 사람을 조회

판다스 활용

- 데이터프레임 데이터 분석

- 데이터 조회

특정 문자열을 포함하는 문자열 데이터 조회

```
df[df['이름'].str.contains('봄')]
```

실행 결과

	이름	성별	나이	키
4	이새봄	여자	28	160

- 이름에 '봄'이 들어간 사람을 조회

판다스 활용

- 데이터프레임 데이터 분석

- 데이터프레임 통계

- 나이와 키 데이터가 숫자형인 데이터프레임 df에 describe() 함수를 사용

데이터프레임 통계 확인

```
df.describe( )
```

실행 결과

	나이	키
count	8.000000	8.000000
mean	25.000000	169.750000
std	2.878492	10.552589
min	21.000000	157.000000
25%	23.750000	161.500000
50%	24.000000	167.500000
75%	26.500000	179.750000
max	30.000000	183.000000

판다스 활용

- 데이터프레임 데이터 분석

- 데이터 갱신

- 행 인덱스나 열 이름으로 데이터프레임 데이터를 조회하고 수정할 수 있음

인덱스로 조회한 데이터를 수정

```
df.loc[4, '키'] = df.loc[4, '키'] + 5  
df.loc[[4]]
```

실행 결과

	이름	성별	나이	키
4	이새봄	여자	28	165

- 인덱스 4번 행의 키 열 값을 5만큼 증가 시킨다

판다스 활용

● 데이터프레임 데이터 분석

• 데이터 갱신

- 반복 연산자(*)를 사용하여 명령을 반복 실행할 수 있음

반복 연산자 사용

```
df.loc[1:3,'키'] = ['모름'] * 3  
df
```

실행 결과

	이름	성별	나이	키
0	허준호	남자	30	183
1	이가원	여자	24	모름
2	배규민	남자	23	모름
3	고고림	남자	21	모름
4	이새봄	여자	28	160
5	이보람	여자	26	163
6	이루리	여자	24	157
7	오다현	여자	24	172

- 인덱스 1번 행부터 3번 행까지 키 데이터를 '모름'으로 한꺼번에 변경

판다스 활용

● 데이터프레임 데이터 분석

- 데이터프레임 구조 수정

- set_index() 함수로 중복 데이터가 없는 이름 열을 행 인덱스로 지정

인덱스 변경(1)

```
df.set_index('이름', inplace=True)
df.head(3)
```

실행 결과

	성별	나이	키
이름			
허준호	남자	30	183
이가원	여자	24	162
배규민	남자	23	179

- inplace=True를 지정하면 작업을 데이터프레임 원본에 적용

판다스 활용

● 데이터프레임 데이터 분석

- 데이터프레임 구조 수정
 - 나이에 10000을 곱한 만큼 보너스

열 생성

```
df['보너스'] = df['나이'] * 10000  
df.head(3)
```

실행 결과

	성별	나이	키	보너스
이름				
허준호	남자	30	183	300000
이가원	여자	24	162	240000
배규민	남자	23	179	230000

판다스 활용

- 데이터프레임 데이터 분석

- 데이터프레임 구조 수정

- 추가한 보너스 열을 drop() 함수로 삭제

열 삭제

```
df.drop('보너스', axis=1, inplace=True)  
df.head(3)
```

실행 결과

	성별	나이	키
이름			
허준호	남자	30	183
이가원	여자	24	162
배규민	남자	23	179

판다스 활용

- 데이터프레임 데이터 분석

- 데이터프레임 구조 수정

- 인덱스를 기본 인덱스로 복구하고 싶다면 `reset_index()` 함수

인덱스 변경(2)

```
df.reset_index(inplace=True)
df.head(3)
```

실행 결과

	이름	성별	나이	키
0	허준호	남자	30	183
1	이가원	여자	24	162
2	배규민	남자	23	179

판다스 활용

- 데이터프레임 데이터 분석

- 데이터프레임 구조 수정

- 데이터를 다른 값으로 치환할 때 `replace()` 함수

데이터 치환

```
rep_cond = {'성별': {'남자': 1, '여자': 0}}  
df2 = df.replace(rep_cond)  
df2.head(3)
```

실행 결과

	이름	성별	나이	키
0	허준호	1	30	183
1	이가원	0	24	162
2	배규민	1	23	179

- 성별이 남자일 때 1, 여자일 때 0으로 치환한 데이터프레임을 `df2`로 저장

판다스 활용

● 데이터프레임 데이터 분석

- 데이터프레임 구조 수정
 - 성별을 기준으로 조회하여 키의 평균과 표준편차를 계산

데이터프레임의 값 연산

#성별 평균 키

```
mean_by_gender = df.groupby(by=['성별'], as_index=False)['키'].mean( )  
mean_by_gender.rename(columns={'키':'평균 키'}, inplace=True)
```

#성별 키의 표준편차

```
std_by_gender = df.groupby(by=['성별'], as_index=False)['키'].std( )  
std_by_gender.rename(columns = {'키':'키의 표준편차'}, inplace=True)
```

```
new_df = pd.merge(mean_by_gender, std_by_gender)  
new_df
```

	성별	평균 키	키의 표준편차
0	남자	181.333333	2.081666
1	여자	162.800000	5.630275

판다스 활용

● 데이터프레임 데이터 분석

• 데이터프레임 구조 수정

- 성별을 기준으로 그룹화하고 그룹별 키의 평균을 계산, 계산 결과는 새로운 데이터프레임 mean_by_gender에 할당
- rename() 함수로 mean_by_gender 데이터프레임에서 키 열의 열 이름을 '평균 키'로 변경
- 성별을 기준으로 그룹화하고 그룹별 키의 표준편차를 계산합니다. 계산 결과는 std_by_gender에 할당
- 데이터프레임 std_by_gender에서 키 열의 열 이름을 '키의 표준 편차'로 변경
- 두 데이터프레임 mean_by_gender와 std_by_gender를 병합